

```

pk |      tag      | parent
----+-----+-----
  1 | alpine linux |
(1 row)

```

```

forumdb=> select * from new_tags ;
pk |      tag      | parent
----+-----+-----
  1 | alpine linux |
(1 row)

```

Now let's see what happens if a record changes its tag from alpine linux to bsd unix:

```

forumdb=> update new_tags set tag='bsd unix' where tag='alpine linux';
move_record
-----

```

```

(1 row)

```

```

UPDATE 0

```

```

forumdb=> select * from new_tags ;
pk |   tag   | parent
----+-----+-----
  1 | bsd unix |
(1 row)

```

```

forumdb=> select * from new_a_tags ;
pk | tag | parent
----+---+-----
(0 rows)

```

```

forumdb=> select * from new_b_tags ;
pk |   tag   | parent
----+-----+-----

```

```
1 | bsd unix |  
(1 row)
```

The rule works! In this short exercise, we have tried to introduce an example of complete rule management. It is a didactic example, and there are many other ways to achieve the same goal. In the next section, we will explore another way to manage events in PostgreSQL: triggers.

Managing triggers in PostgreSQL

In the previous section, we talked about rules. In this section, we will talk about triggers, what they are, and how to use them. We need to start by understanding what triggers are; if we understand what rules are, this should be simple. In the previous section, we defined rules as simple event handlers; now we can define triggers as complex event handlers. For triggers, as for rules, there are NEW and OLD records, which assume the same meaning for triggers as they did for rules. For triggers, the manageable events are INSERT/DELETE/UPDATE and TRUNCATE. Another difference between rules and triggers is that with triggers, it is possible to handle INSERT/UPDATE/DELETE and TRUNCATE events before they happen or after they have happened. With triggers, we can also use the INSTEAD OF option, but only on views.

So, we can manage the following events:

- BEFORE INSERT/UPDATE/DELETE/TRUNCATE
- AFTER INSERT/UPDATE/DELETE/TRUNCATE
- INSTEAD OF INSERT/UPDATE/DELETE

With rules, it is possible to have only the NEW record for INSERT operations, the NEW and OLD record for UPDATE operations, and the OLD record for DELETE operations. The first two list items can also be used on foreign tables as well as real tables, and the third list item can only be used on views. For further information, see <https://www.postgresql.org/docs/current/sql-createtrigger.html>.

We will now take the first steps to use triggers, and we will find out how to obtain the same results that are achieved when using rules. With triggers, we can do everything we can do with rules and much more.

Before continuing, we need to keep two things in mind:

- If triggers and rules are simultaneously present on the same event in a table, the rules always fire before the triggers.

- If there are multiple triggers on the same event of a table (for example, BEFORE INSERT), they are executed in alphabetical order.

There is another category of triggers, called event triggers, which will be covered in the *Event triggers* section.

Trigger syntax

As described in the official document, the syntax for defining a trigger is as follows:

```
CREATE [ CONSTRAINT ] TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event
[ OR ... ] }
ON table_name
[ FROM referenced_table_name ]
[ NOT DEFERRABLE | [ DEFERRABLE ] [ INITIALLY IMMEDIATE | INITIALLY
DEFERRED ] ]
[ REFERENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [
... ] ]
[ FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE { FUNCTION | PROCEDURE } function_name ( arguments )

where event can be one of:

INSERT
UPDATE [ OF column_name [, ... ] ]
DELETE
TRUNCATE
```

We will only look at the most used aspects of this syntax; for further information, see <https://www.PostgreSQL.org/docs/current/sql-createtrigger.html>. The key points behind the execution of a trigger are as follows:

- The event that we want to handle, for example, INSERT, DELETE, or UPDATE.
- When we want the TRIGGER execution to start (for example, BEFORE INSERT).
- The trigger calls a function to perform some action.

The function invoked by the trigger must be defined in a particular way, as shown in the prototype here:

```
CREATE OR REPLACE FUNCTION function_name RETURNS trigger as
$$
DECLARE
....
BEGIN

    RETURN
END;
$$
LANGUAGE 'plpgsql';
```

The functions that are called by the triggers are functions that have no input parameters and must return a TRIGGER type; these functions take the parameters from the NEW/OLD records. Starting with this prototype of the preceding function, a possible TRIGGER definition of the BEFORE INSERT event can be described as follows:

```
CREATE TRIGGER trigger_name BEFORE INSERT on table_name FOR EACH ROW
EXECUTE PROCEDURE function_name.
```

There is also this syntax:

```
CREATE TRIGGER trigger_name BEFORE INSERT on table_name FOR EACH STATEMENT
EXECUTE PROCEDURE function_name.
```

The difference between FOR EACH ROW and FOR EACH STATEMENT is that:

- A trigger defined with FOR EACH ROW is executed for each row involved in the operation (for example, for each row inserted, updated, or deleted) that satisfies the condition of the trigger.
- A trigger defined with FOR EACH STATEMENT is executed only once for each SQL statement that satisfies the trigger's condition, no matter how many rows are involved in the operation.

In the next section, we will try to implement what we wrote with the rules, this time applying triggers.

Triggers on INSERT

In this section, we will see how to make our first triggers:

1. Let's go back to the rule that we wrote in the *ALSO option* section; we wrote a rule like this:

```
create or replace rule r_tags1
as on INSERT to tags
where NEW.tag like 'a%' DO ALSO
insert into new_a_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
```

2. Now let's see how we can achieve the same goal using a trigger. First, let's go back to the initial situation:

```
forumdb=> drop table if exists new_tags cascade;
forumdb=> create table new_tags as select * from tags limit 0;
forumdb=> truncate table new_a_tags;
```

3. Now we can create the function, which will then be called by the trigger:

```
forumdb=> CREATE OR REPLACE FUNCTION f_tags() RETURNS trigger as
$$
BEGIN
  IF lower(substring(NEW.tag from 1 for 1)) = 'a' THEN
    insert into new_a_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
  END IF;
  RETURN NEW;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION
```

Let's take a deeper look at what the code means:

- The statement `lower(substring (NEW.tag from 1 for 1))` takes the first character of a string and converts it into lowercase.
- The `RETURN NEW` statement passes the new record from the table to the `INSERT` in the `new_tags` table.

4. Now let's define the trigger on the BEFORE INSERT event of the t_tags table:

```
forumdb=> CREATE TRIGGER t_tags BEFORE INSERT on new_tags FOR EACH
ROW EXECUTE PROCEDURE f_tags();
CREATE TRIGGER
```

5. So, when a value is inserted into the new_tags table, before executing the INSERT, the trigger is executed and returns the NEW record to the default action (INSERT on the new_tags table). Now let's check that it works:

```
forumdb=> insert into new_tags (pk,tag,parent) values (1,'bsd
unix',NULL);
INSERT 0 1

forumdb=> insert into new_tags (pk,tag,parent) values (2,'alpine
linux',1);
INSERT 0 1

forumdb=> select * from new_tags ;
pk |      tag      | parent
----+-----+-----
  1 | bsd unix      |
  2 | alpine linux  |      1
(2 rows)

forumdb=> select * from new_a_tags ;
pk |      tag      | parent
----+-----+-----
  2 | alpine linux  |      1
(1 row)
```

As we can see here, it works!

6. We will proceed from here, step by step, to better understand the difference between working with rules and working with triggers. The goal we want to achieve with triggers is to receive the same result as what we can achieve with the following rule:

```
create or replace rule r_tags2
as on INSERT to tags
```

```

where NEW.tag ilike 'b%'
DO INSTEAD insert into new_b_tags(pk,tag,parent)values (NEW.pk,NEW.
tag,NEW.parent);

```

7. For now, let's use the same procedure we used in the rules by creating a new function, which will then be fired from the trigger:

```

forumbd=> CREATE OR REPLACE FUNCTION f2_tags() RETURNS trigger as
$$
BEGIN
  IF lower(substring(NEW.tag from 1 for 1)) = 'b' THEN
    insert into new_b_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
  RETURN NULL;
  END IF;
  RETURN NEW;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION

forumbd=> CREATE TRIGGER t2_tags BEFORE INSERT on new_tags FOR EACH
ROW EXECUTE PROCEDURE f2_tags();
CREATE TRIGGER

```

8. The lower statement, `(substring(NEW.tag from 1 for 1)) = 'b'`, is practically identical to what we first saw in relation to rules. The difference is the `RETURN NULL`, which means that if the `NEW.tag` value starts with `b`, then a `NULL` value is returned to the default action and the `INSERT` on the `new_tags` table will not insert any value. If, instead, the `IF` condition is not satisfied, then the function returns `NEW` and the record is inserted into the `new_tags` table.

Let's see if it works:

```

forumbd=> truncate new_tags;
TRUNCATE TABLE
forumbd=> truncate new_a_tags;
TRUNCATE TABLE
forumbd=> truncate new_b_tags;

```

```
TRUNCATE TABLE
```

```
forumbd=> insert into new_tags (pk,tag,parent) values (1,'bsd
unix',NULL);
INSERT 0 0
```

As we can see, the IF condition works, and the result, INSERT 0 0, means that no record has been inserted into the new_tags table. This happened because the trigger works on the BEFORE INSERT event and the IF condition moved the record to the new_b_tags table.

9. We will now look at how to write the whole procedure using a single trigger. First, let's go back to the initial conditions of our environment. As before, we delete the data in the tables and, using the CASCADE option, we delete the selected trigger and all the triggers associated with it:

```
forumbd=> TRUNCATE new_tags;
TRUNCATE TABLE
forumbd=> TRUNCATE new_a_tags;
TRUNCATE TABLE
forumbd=> TRUNCATE new_b_tags;
TRUNCATE TABLE
forumbd=> DROP TRIGGER t_tags ON new_tags CASCADE;
DROP TRIGGER
forumbd=> DROP TRIGGER t2_tags ON new_tags CASCADE;
DROP TRIGGER
```

10. In this last step, we will combine what we have written in the functions f_tags () and f2_tags() into a single function, f3_tags(), which will be fired from the t3_tags trigger on the event BEFORE INSERT:

```
forumbd=> CREATE OR REPLACE FUNCTION f3_tags() RETURNS trigger as
$$
BEGIN
  IF lower(substring(NEW.tag from 1 for 1)) = 'a' THEN
    insert into new_a_tags(pk,tag,parent)values (NEW.pk,NEW.
tag,NEW.parent);
    RETURN NEW;
  ELSIF lower(substring(NEW.tag from 1 for 1)) = 'b' THEN
```

```

        insert into new_b_tags(pk,tag,parent)values (NEW.pk,NEW.
tag,NEW.parent);
        RETURN NULL;
    ELSE
        RETURN NEW;
    END IF;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION

forumdb=> CREATE TRIGGER t3_tags BEFORE INSERT on new_tags FOR EACH
ROW EXECUTE PROCEDURE f3_tags();
CREATE TRIGGER

```

This function contains the logic of the two functions previously seen. This way, we can solve the problem more elegantly by using a single function and a single trigger. Let's see if it works:

```

forumdb=> insert into new_tags (pk,tag,parent) values (1,'operating
systems',NULL);
INSERT 0 1

forumdb=> insert into new_tags (pk,tag,parent) values (2,'alpine
linux',1);
INSERT 0 1
forumdb=> insert into new_tags (pk,tag,parent) values (3,'bsd
unix',1);
INSERT 0 0

forumdb=> select * from new_tags ;
pk |          tag          | parent
----+-----+-----
  1 | operating systems |
  2 | alpine linux      |      1
(2 rows)

forumdb=> select * from new_a_tags ;

```

```

pk |      tag      | parent
----+-----+-----
  2 | alpine linux |      1
(1 row)

forumdb=> select * from new_b_tags ;
pk |   tag   | parent
----+-----+-----
  3 | bsd unix |      1
(1 row)

```

As can be seen, the function works.

The TG_OP variable

As shown in the official documentation at <https://www.PostgreSQL.org/docs/current/plpgsql-trigger.html>, control of the triggers in PostgreSQL is allowed using special variables, two of which we have already seen (the NEW variable and the OLD variable). There is another special variable called TG_OP, which tells us from which event the trigger is fired. The possible values of the TG_OP variable are INSERT, DELETE, UPDATE, and TRUNCATE.

Triggers on UPDATE / DELETE

Now, let's go back to the example we used in *Figure 8.1*. The goal we want to achieve is to create a single function that is able to handle the INSERT, DELETE, and UPDATE events. First, let's return to the initial conditions in our environment:

```

forumdb=> truncate new_tags;
TRUNCATE TABLE
forumdb=> truncate new_a_tags;
TRUNCATE TABLE
forumdb=> truncate new_b_tags;
TRUNCATE TABLE
forumdb=> drop trigger t3_tags on new_tags cascade;
DROP TRIGGER

```

Now, as before, we will proceed step by step. The first step is to write the section of code that will be performed during the INSERT event. Then, we will see how to extend the function to manage the DELETE and UPDATE events.